

AGENTIC AI ENGINEER

The Six-Week Roadmap

A structured, week-by-week plan from basic programming to a shipped, observable AI agent — grounded in the tools and practices used in production agentic systems today.

Week	Focus
Week I	Programming Foundations
Week II	How LLMs Actually Work
Week III	Agentic Frameworks & Prompt Engineering
Week IV	RAG & Vector Databases
Week V	Memory & Tool Use (MCP)
Week VI	Capstone Project & Observability

Programming Foundations

Get comfortable with the basic building blocks before touching any AI framework

OVERVIEW

This week is about removing friction, not achieving mastery. Every agent framework you will meet later is, underneath, ordinary Python code that calls web APIs and passes JSON back and forth, some of it waiting on network calls instead of blocking your whole program. If you can read that kind of code comfortably, you are ready for Week 2.

TOPICS TO COVER THIS WEEK

1	Python Basics Variables, data types, loops, conditionals, functions, and just enough object-oriented programming (classes and objects) to understand how frameworks structure their code.
2	Working with APIs What a REST API is, how to call one with Python's requests library, and how to read status codes, headers, and response bodies.
3	JSON Reading and writing JSON — the universal format agents use to exchange structured data between models, tools, and services.
4	Async Calls What async and await mean in Python, and why agents frequently run several tasks — an API call, a database lookup, a tool call — concurrently instead of one after another.

Why this matters: Every agent framework you will study from Week 3 onward is built directly on top of Python, HTTP APIs, and JSON. Skipping this foundation is the single biggest reason beginners feel lost when they open their first LangChain or LangGraph codebase.

WEEK I — IN DETAIL

Programming Foundations

THIS WEEK, IN DETAIL

Python & OOP in agent code

You do not need advanced Python — but you will constantly see classes with names like `Agent`, `Tool`, or `Chain`. Understanding that a class simply bundles data (attributes) with behaviour (methods) is enough to read almost any framework's source.

APIs as the nervous system of an agent

An agent's 'senses' and 'hands' are almost always APIs — a weather API, a search API, a database API. Learn to make a GET and a POST request, read a JSON response, and handle a failed request (a non-200 status code) gracefully.

JSON as the universal contract

Nearly every tool call, every LLM response schema, and every configuration file in the agentic-AI world is JSON. Practice converting a Python dictionary to JSON and back until it feels automatic.

Why async matters for agents

A real agent often waits on several slow things at once: a model response, a database query, a web search. Synchronous code would make it wait for each one in turn; async code lets it wait for all of them at the same time, which is why almost every production agent framework is built on Python's `asyncio`.

TOOLS & LIBRARIES TO KNOW

- Python 3.11+
- `requests` / `httpx`
- `json` (standard library)
- `asyncio`
- Postman or `curl` (for testing APIs by hand)

PRACTICAL EXERCISE

Write a script that calls any free public API (a weather or currency-exchange API works well), parses the JSON response, and prints two or three fields from it. Then rewrite the same script using `async/await` and `aiohttp`, and call the API three times concurrently instead of in sequence — compare how much faster it runs.

END-OF-WEEK CHECKLIST

- [] Comfortably read and write basic Python functions and classes
- [] Call a REST API and handle its JSON response
- [] Explain the difference between synchronous and asynchronous code in one sentence

How LLMs Actually Work

Look inside the engine that powers every agent, without needing the underlying math

OVERVIEW

Build an intuitive, working understanding of what a Large Language Model is doing when it generates a response. You do not need to derive the attention mechanism from first principles — you need to know what tokens, context windows, and training phases are, and why they explain so many of the quirks you will see later.

TOPICS TO COVER THIS WEEK

1	Transformer Basics The architecture behind almost every modern LLM, understood at a conceptual level: how 'attention' lets a model weigh which earlier words matter most when predicting the next one.
2	Tokens How text is broken into tokens (often sub-word pieces, not whole words), and why the number of tokens — not the number of characters — determines cost and speed.
3	Context Window The maximum number of tokens a model can 'see' at once, and why information placed very early in a long conversation can effectively fall out of view.
4	Training Phases Pre-training on vast text corpora, followed by fine-tuning and alignment (often via human feedback, commonly called RLHF), which is what turns a raw next-token predictor into a helpful, instruction-following assistant.

Why this matters: Prompt engineering, RAG, and memory management — everything you learn from Week 3 onward — are really just different strategies for working around the limits of tokens and context windows. Understanding this week turns those later topics from 'rules to memorise' into 'consequences that make obvious sense'.

WEEK II — IN DETAIL

How LLMs Actually Work

THIS WEEK, IN DETAIL

Attention, in plain language

A transformer does not read a sentence strictly left to right the way older models did. For every word it generates, it looks back across the entire input and decides which earlier words to pay the most 'attention' to. This is what lets a model connect a pronoun to the correct noun several sentences earlier.

Tokens and why they matter for cost

Tokenizers typically split text into sub-word units, so a single unusual word may become two or three tokens. Providers charge and rate-limit by token count, and libraries such as tiktoken let you count tokens locally before sending a request, which is essential for controlling cost in an agent that runs thousands of times a day.

Context windows keep growing, but are not free

Frontier models in 2026 commonly offer context windows in the hundreds of thousands of tokens, and some go beyond a million. A larger window does not mean the model uses every token equally well — recall of information in the middle of a very long context is still measurably weaker than information near the start or end, an effect often called 'lost in the middle'.

From raw model to helpful assistant

A freshly pre-trained model is simply a very good next-token predictor; it is not yet an assistant. Fine-tuning on curated instruction-response pairs, followed by an alignment stage that uses human feedback to reward helpful and honest answers, is what shapes it into something you can reliably converse with and give instructions to.

TOOLS & LIBRARIES TO KNOW

- tiktoken (token counting)
- Hugging Face Transformers (for hands-on exploration)
- Any frontier model's official documentation on context-window limits

PRACTICAL EXERCISE

Take a long article (2,000+ words) and count its tokens with tiktoken. Then paste it into a conversation with an LLM, bury one specific fact in the middle of it, and ask a question that requires recalling that fact. Repeat with the fact placed at the very start and the very end, and compare how reliably the model answers each time.

END-OF-WEEK CHECKLIST

- [] Explain what a token is and why token count drives cost
- [] Describe what a context window limits, in one or two sentences
- [] Name the two broad phases a raw LLM goes through before it becomes an assistant

Agentic Frameworks & Prompt Engineering

Learn the tools that turn a plain LLM into a working, tool-using agent

OVERVIEW

Get hands-on with the frameworks most teams actually use in 2026 to wire an LLM into an agent, and learn to write prompts that get consistent, reliable results instead of occasionally-brilliant, occasionally-broken ones.

TOPICS TO COVER THIS WEEK

1	LangChain A general-purpose framework for LLM applications, built around composable components — prompts, models, retrievers, and tools — with the largest integration ecosystem in the space.
2	LangGraph A graph-based orchestration framework from the LangChain team, purpose-built for stateful, multi-step agents: you compose nodes (an LLM call or a tool call) connected by edges that route based on the current state.
3	LlamaIndex A framework that started with a narrower, sharper focus: connecting your own documents and data to an LLM through strong indexing and retrieval.
4	Prompt Engineering Writing clear, structured prompts: zero-shot and few-shot examples, chain-of-thought reasoning, structured (JSON-schema) outputs, and breaking a complex task into a chain of simpler prompts.

***Why this matters:** Frameworks save you from reinventing orchestration logic that hundreds of teams have already solved, and disciplined prompting is often the difference between an agent that works four times out of five and one that works reliably enough to trust in production.*

WEEK III — IN DETAIL

Agentic Frameworks & Prompt Engineering

THIS WEEK, IN DETAIL

Choosing between LangChain, LangGraph, and LlamaIndex

In practice these are no longer rival choices so much as complementary layers. LangChain (through LangGraph) is generally regarded as the stronger foundation for complex, multi-tool, stateful agent workflows, while LlamaIndex is generally regarded as stronger for document-heavy retrieval. A large share of production systems in 2026 quietly use both together — LlamaIndex for the retrieval layer, LangGraph for the orchestration layer.

Why LangGraph over a simple chain

A simple linear 'chain' works fine while your agent does one thing after another. The moment you need branching logic, retries, human-in-the-loop approval, or an agent that can loop back and try again, a graph of nodes and edges — which is exactly what LangGraph provides — becomes far easier to reason about and debug than a long chain of nested function calls.

Prompting techniques worth knowing

Zero-shot prompting simply asks directly; few-shot prompting shows two or three worked examples first. Chain-of-thought prompting — asking the model to reason step by step — reliably improves accuracy on arithmetic and multi-step logic problems, though on newer 'reasoning' models this instruction is often unnecessary and can occasionally even hurt accuracy, since the model already reasons internally. Structured-output prompting, where you specify an exact JSON schema for the response, is one of the most reliable techniques for agents, because it turns a fuzzy natural-language answer into something your code can parse safely.

Prompt chaining and repetition

Breaking one large task into a sequence of smaller prompts — where each step's output feeds the next — mirrors how a person would tackle a complex problem, and tends to be more reliable than asking for everything in a single giant prompt. Repeating a critical constraint explicitly (for example, stating a formatting rule twice) is a small trick that measurably improves how consistently a model follows it.

TOOLS & LIBRARIES TO KNOW

- LangChain
- LangGraph
- LlamaIndex
- OpenAI / Anthropic prompt-design documentation

PRACTICAL EXERCISE

Pick one framework — LangGraph is a good default — and build a small agent with a single tool (for example, a calculator function or a simple web-search wrapper). Force its final answer into a strict JSON schema, and test what happens to reliability when you add a one-line chain-of-thought instruction versus removing it.

END-OF-WEEK CHECKLIST

- [] Explain the difference between a simple chain and a LangGraph-style stateful agent
- [] Write a prompt that reliably returns a specific JSON structure
- [] Describe when chain-of-thought prompting helps and when it is unnecessary

RAG & Vector Databases

Give your agent knowledge beyond what it learned during training

OVERVIEW

Understand, end to end, how an agent retrieves relevant information from your own documents before answering — and why the way you split those documents into chunks quietly decides most of your retrieval quality.

TOPICS TO COVER THIS WEEK

1	Retrieval-Augmented Generation (RAG) The overall pipeline: retrieve the most relevant pieces of your own data, then hand them to the LLM alongside the user's question so it can generate a grounded answer.
2	Vector Databases Purpose-built stores such as Pinecone, Chroma, Weaviate, or FAISS that index data by meaning rather than by exact keyword match.
3	Embeddings How a piece of text is converted into a numeric vector that captures its meaning, so that semantically similar text ends up 'close together' in vector space.
4	Chunking Splitting large documents into smaller, coherent pieces before embedding them — arguably the single most underrated lever in a RAG system's accuracy.
5	Retrieval Components Retrievers that fetch an initial set of candidate chunks, and rerankers that reorder them by relevance before they reach the model.

Why this matters: RAG is what lets an agent answer questions about your own company's documents, your product's manuals, or your own notes, instead of being limited to whatever it happened to learn during training. A well-chunked, well-retrieved pipeline is often the difference between an agent that sounds confident and one that is actually correct.

THIS WEEK, IN DETAIL

Why chunking decides more than the embedding model

Independent benchmarking has found that the chunking configuration you choose can influence retrieval quality as much as, or more than, the embedding model itself — getting the chunk size wrong by even one size class can measurably hurt precision. The reason is simple: an embedding model compresses an entire chunk into a single vector, so if that chunk mixes two unrelated ideas, the resulting vector represents neither one well.

A sensible default, and when to deviate

Recursive, structure-aware splitting at roughly 400 to 512 tokens with 10 to 20 percent overlap is the widely recommended starting point for most content, and is the default behaviour of LangChain's RecursiveCharacterTextSplitter. From there, the right strategy is really a domain question rather than a library choice: semantic chunking (splitting at topic boundaries rather than fixed sizes) tends to help most on long-form prose, document-structure-aware chunking matters most for PDFs, legal contracts, or Markdown with headings, and page-level chunking has performed strongly on cleanly paginated documents.

Embeddings and similarity search

An embedding model turns text into a vector of a few hundred to a few thousand numbers. A vector database's job is to store millions of these vectors and quickly find the ones closest to your query's vector — this 'closeness' is what stands in for semantic similarity, and it is why a RAG system can retrieve a relevant passage even when it does not share a single exact keyword with the question.

Retrieval is rarely the last step

Most production RAG pipelines do not stop at the first retrieval pass. A reranker — often a smaller, more precise model — takes the top candidates from the initial retrieval and reorders them by true relevance to the specific question, which noticeably improves answer quality before anything reaches the LLM.

TOOLS & LIBRARIES TO KNOW

- Chroma / FAISS (good for learning)
- Pinecone / Weaviate (production-scale options)
- LangChain's RecursiveCharacterTextSplitter
- A cross-encoder reranker (e.g. from Sentence-Transformers)

PRACTICAL EXERCISE

Take one PDF (your own notes work well) and build a small RAG pipeline: chunk it with a recursive splitter at around 500 tokens, embed the chunks, store them in Chroma or FAISS, and answer three questions using only retrieved chunks. Then re-chunk the same document at a much larger and a much smaller size, and compare answer quality.

END-OF-WEEK CHECKLIST

- [] Explain the RAG pipeline in your own words, start to finish
- [] Describe why chunk size and overlap affect retrieval quality
- [] Build and query a small vector store from a real document

Memory & Tool Use (MCP)

Make your agent remember what matters, and act on the world through tools

OVERVIEW

Learn how agents keep track of context across a conversation and across sessions, and how the Model Context Protocol (MCP) has become the standard way agents connect to external tools and data in 2026.

TOPICS TO COVER THIS WEEK

1	Short-Term Memory How an agent keeps track of what has already happened earlier in the same conversation, so it does not contradict or forget itself mid-session.
2	Long-Term Memory Persisting information across sessions — user preferences, past decisions, prior facts — so the agent 'remembers' you the next time you return.
3	MCP (Model Context Protocol) An open standard, introduced by Anthropic in late 2024, that gives any AI model one consistent way to connect to external tools, data sources, and APIs.
4	Tool Calling How an LLM decides, mid-response, to call a specific function or tool, and how the tool's result is passed back into the conversation for the model to use.
5	Tool Usage Patterns Practical judgement about when and how much capability to hand an agent — the difference between a tool that reads data safely and one that can take irreversible action.

Why this matters: Memory and tool use are what separate a simple chatbot from a genuine agent — one that can recall context, reach real systems, and actually get something done rather than only describe what it would do.

THIS WEEK, IN DETAIL

Why MCP exists

Before MCP, every model needed a custom, hand-built integration for every tool it used — if you wanted both Claude and another model to read the same Notion workspace, you needed two separate integrations. MCP replaces that N-times-M integration problem with one shared protocol: any MCP-compatible model can speak to any MCP-compatible tool without a bespoke connector.

How MCP is structured

MCP defines three roles. The host is the AI application itself — an agent, Claude Desktop, or a custom application. The client lives inside the host and manages communication with a specific server. Each server is a focused adapter for one external system — a database, a code repository, a SaaS tool — and exposes its capabilities as tools (callable functions), resources (readable data), and prompts (reusable templates).

Where MCP stands in 2026

Anthropic donated MCP to the Linux Foundation's Agentic AI Foundation in December 2025, with OpenAI, Google DeepMind, Microsoft, AWS, and others backing it, and by 2026 it has become the de facto standard for agent-to-tool communication, with native support across most major model providers and coding tools. A public registry of ready-made MCP servers also means you can often reuse an existing connector instead of building your own.

Memory in practice, and tool-use judgement

Short-term memory is usually just the running conversation history passed back to the model on every turn, sometimes summarised once it grows too long to fit the context window. Long-term memory typically lives in an external store — a database or vector store — that the agent explicitly writes to and retrieves from. On tool use: the wider the agent's capabilities, the more deliberately you should scope what each tool is allowed to do, since an agent that can only read data is a very different risk profile from one that can send money or delete records.

TOOLS & LIBRARIES TO KNOW

- Model Context Protocol SDKs (Python / TypeScript)
- A simple key-value or vector store for long-term memory
- LangGraph's built-in state/checkpointing for short-term memory

PRACTICAL EXERCISE

Build a minimal MCP server that exposes exactly one tool (a calculator or a unit-converter function), connect an agent to it, and confirm the agent calls the tool correctly rather than guessing the answer itself. Separately, give the same agent a simple long-term memory (a small file or database) and confirm it recalls a fact you told it in an earlier, separate session.

END-OF-WEEK CHECKLIST

- [] Explain the difference between short-term and long-term memory in an agent
- [] Describe what MCP standardises and why it matters
- [] Build and call one working tool through an agent

Capstone Project & Observability

Build it, watch it, and make it safe enough to trust

OVERVIEW

Bring everything from Weeks 1 to 5 together into one real, end-to-end agent project, then add the two layers every production AI system needs: observability, so you can see what your agent is actually doing, and guardrails, so it behaves reliably even when the input is messy or adversarial.

TOPICS TO COVER THIS WEEK

1	Capstone Project Combine programming, an LLM, an agentic framework, RAG, and memory or tool use into one working project you can demonstrate end to end.
2	Observability (Langfuse or LangSmith) Tracing and logging every step your agent takes — every model call, every tool call, every retrieval — so failures are debuggable instead of mysterious.
3	Output Verification Checking agent responses for accuracy and catching hallucinations before they reach a user.
4	Guardrails Input and output validation and safety checks — blocking prompt injection, validating structured output, catching PII — so the agent behaves reliably under real-world use.
5	Ship & Share Documenting the finished project clearly, for example in a GitHub repository, so others can understand, run, and build on your work.

Why this matters: A project without observability and guardrails is a demo, not a product. This final week is what turns six weeks of learning into something you can genuinely show off, and — more importantly — something you can trust.

THIS WEEK, IN DETAIL

Choosing an observability tool

Langfuse and LangSmith are the two most widely used options in 2026. Langfuse is open source (MIT-licensed), framework-agnostic, built on OpenTelemetry, and fully self-hostable, which makes it a strong default if you want to avoid vendor lock-in or need to keep data in-house. LangSmith is a proprietary, closed-source platform built by the LangChain team, with the deepest native integration if your stack is already built on LangChain or LangGraph. Either one traces the full path of a request — every model call, retrieval step, and tool invocation — so when an agent gives a wrong answer, you can see exactly which step produced it.

What output verification actually checks

Verification means comparing what the agent said against what is actually true or intended — checking a claimed fact against your source documents, validating that a structured response matches its expected schema, or running an automated 'LLM-as-judge' evaluation against a small test set of known-good answers before you trust a change enough to ship it.

Guardrails: input, output, and everything in between

A mature guardrail setup checks more than one point in the pipeline: an input rail screens the user's message for prompt-injection attempts before it reaches the model; an output rail validates the model's response against a schema and screens it for PII or unsafe content before it reaches the user; and, for agents with tools, a tool-call rail checks that a requested tool call is actually permitted before it runs. Guardrails AI (schema and content validators you compose directly in Python) and NVIDIA's NeMo Guardrails (programmable dialogue rails for controlling what an agent will and will not discuss) are two of the most established open-source options for this.

Publishing your work

A clear README — what the project does, how to run it, what framework and tools it uses, and a short demo — is what turns a personal exercise into something a recruiter, collaborator, or future employer can actually evaluate. This is also the natural place to link back to the rest of this six-week roadmap.

TOOLS & LIBRARIES TO KNOW

- Langfuse or LangSmith
- Guardrails AI or NeMo Guardrails
- A simple eval set of question-answer pairs for regression testing
- GitHub (for publishing the finished project)

PRACTICAL EXERCISE

Take the agent you have been building since Week 3 (with RAG from Week 4 and tools from Week 5), add tracing with Langfuse or LangSmith, and add at least one input guardrail (basic prompt-injection screening) and one output guardrail (schema or PII validation). Deliberately try to break your own agent before you consider it finished — then publish the project, with a short README, to a public GitHub repository.

END-OF-WEEK CHECKLIST

- [] Ship one end-to-end agent project combining at least three prior weeks' skills
- [] Add tracing plus at least one input and one output guardrail
- [] Publish the project with clear documentation